

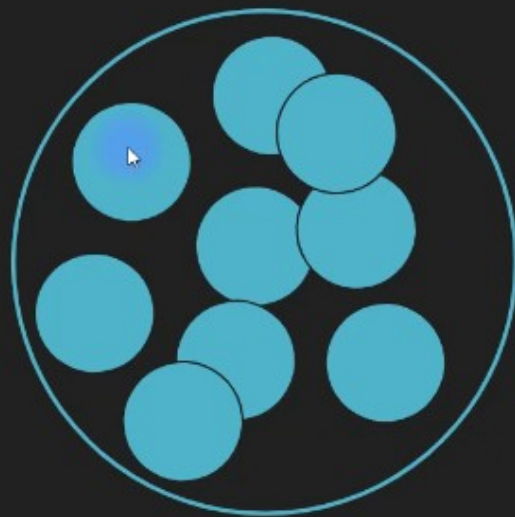
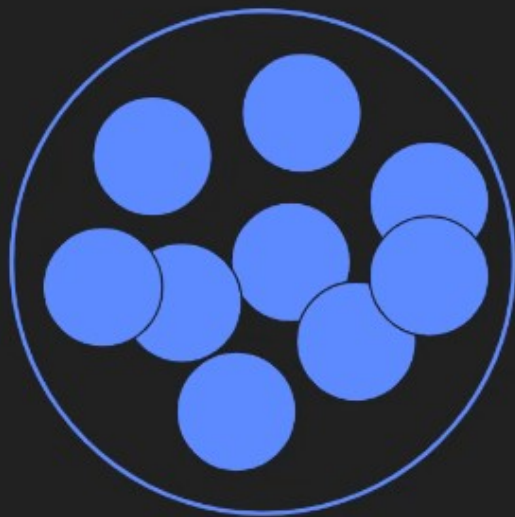
Motivation and description of correlation

In this video, you will learn

- The motivation behind correlation analyses.
- What a correlation coefficient does and does not show.

Beyond the t-test

$H_A = \text{blue} \text{ vs. } \text{red}$

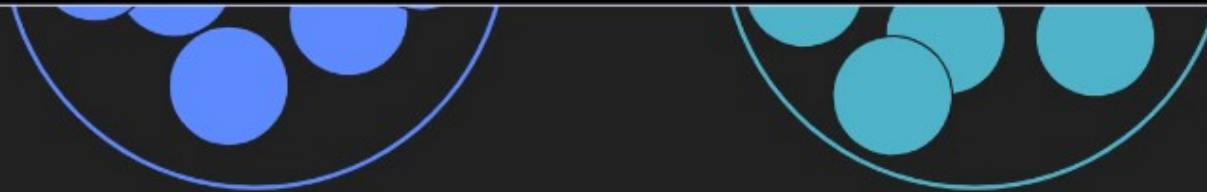


Beyond the t-test

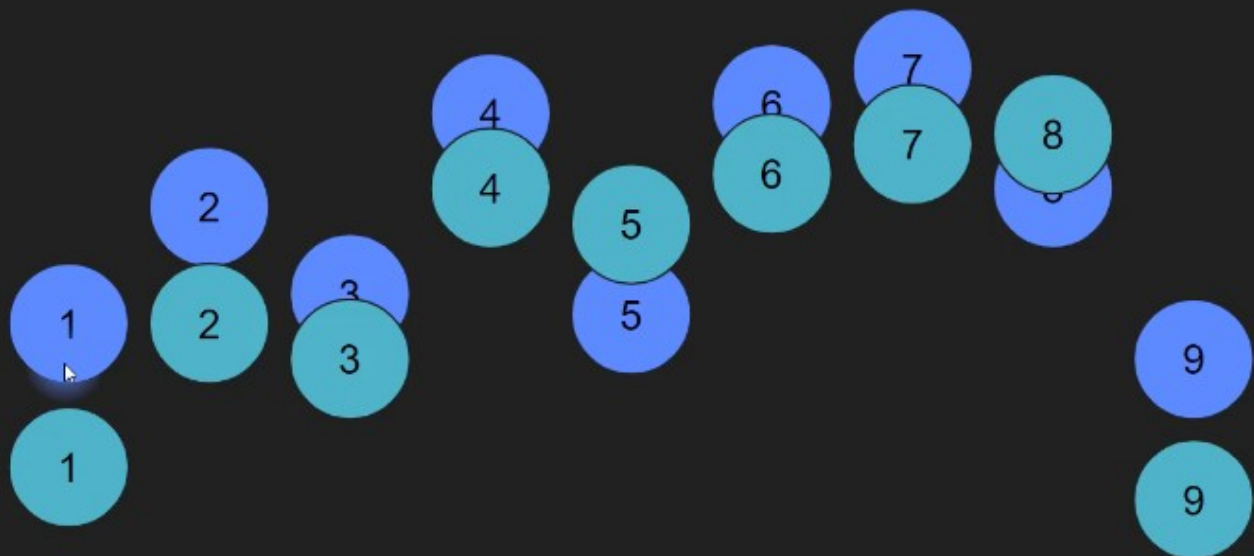
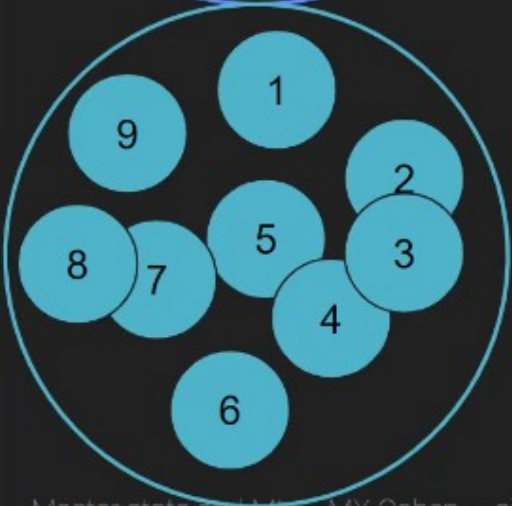
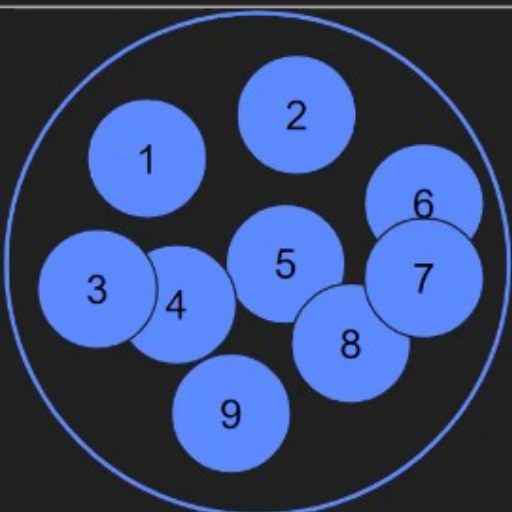
$$H_A = \text{blue circle} \text{ vs. } \text{red circle}$$

“Problem” (bug? feature?):

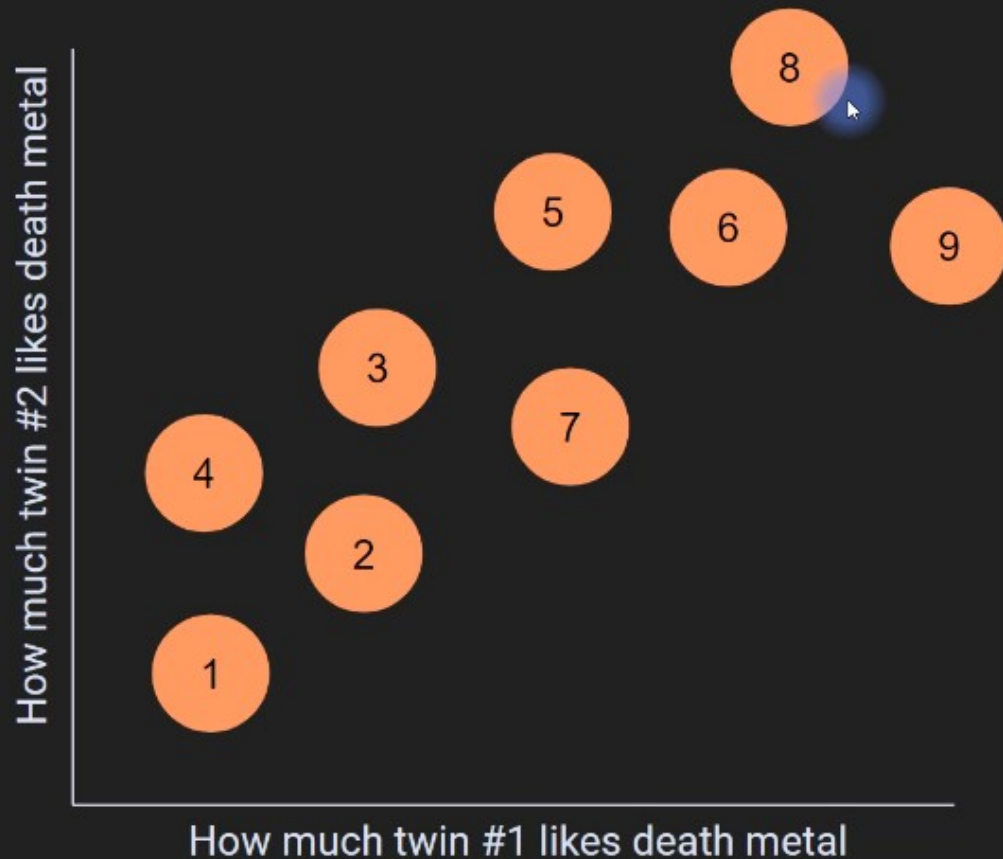
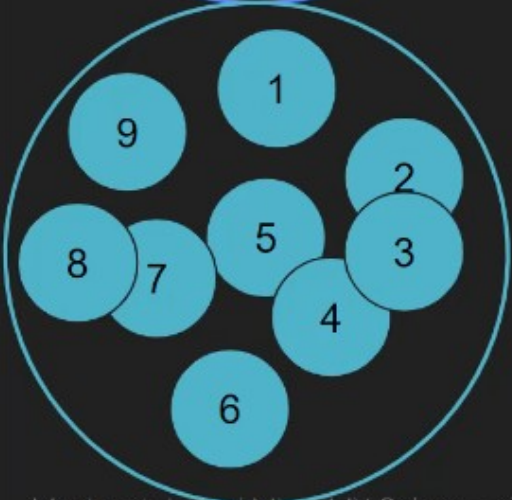
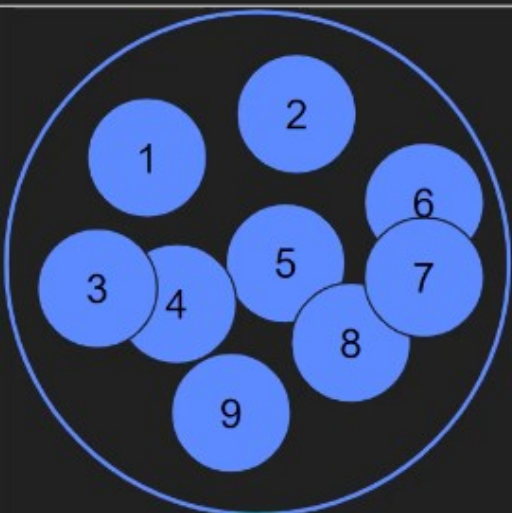
The t-test (and other measures of group differences) ignores individual differences.



Enter the correlation

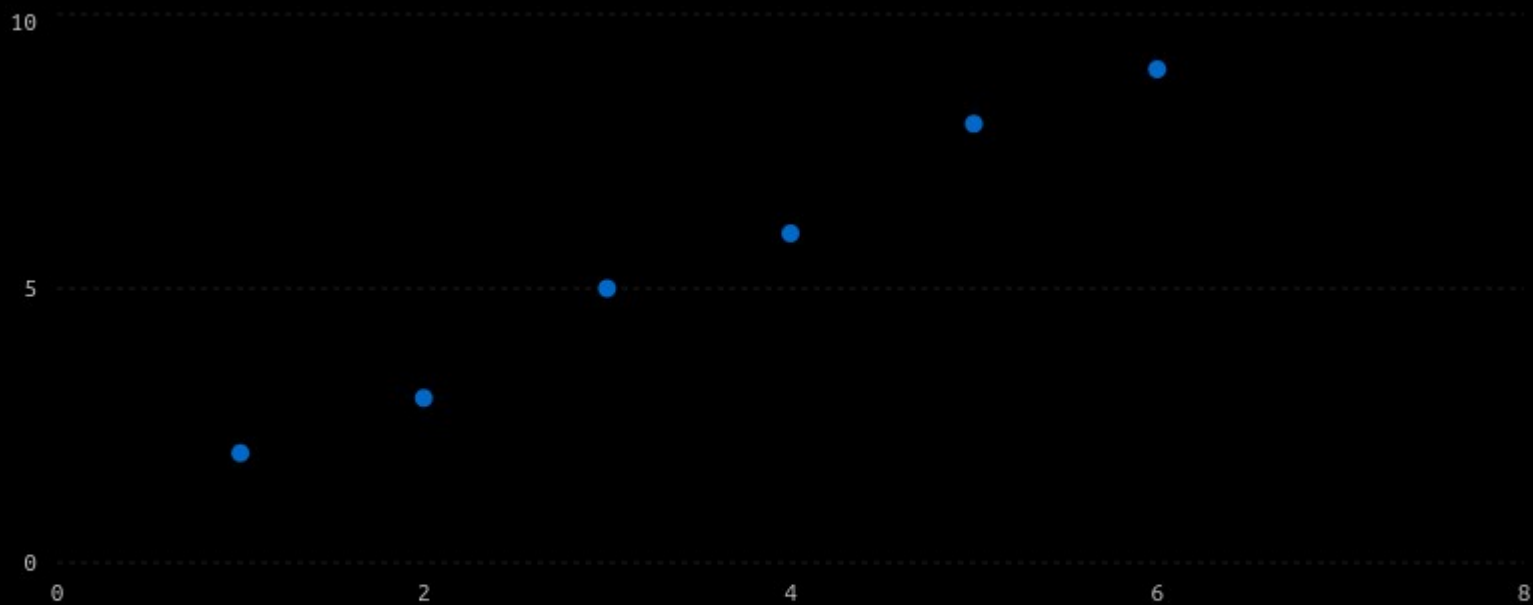


Enter the correlation



Positive Covariance

As X increases, Y also tends to increase.



Example:

- Hours studied $\uparrow$
- Exam score $\uparrow$

Correlation: definition and description

A correlation analysis computes a correlation coefficient (termed “ r ”).

The correlation coefficient r is a single number that shows the linear relationship between two variables.

The correlation coefficient varies between -1 and +1. -1 means a perfect inverse relationship, 0 means no relationship, and +1 means a perfect positive relationship.

The correlation coefficient itself is a continuous measure of correlation strength. A corresponding p-value must be computed to interpret its statistical significance.

Correlation: definition and description

Positive correlation



Negative correlation



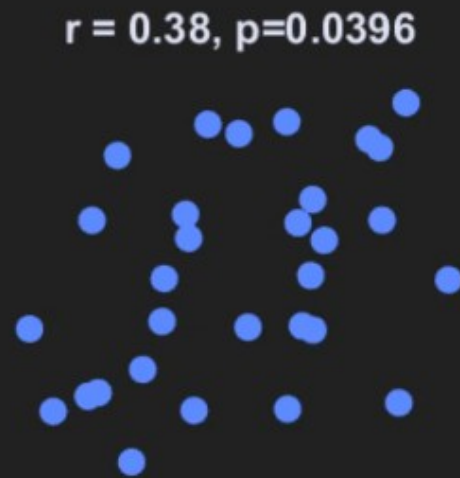
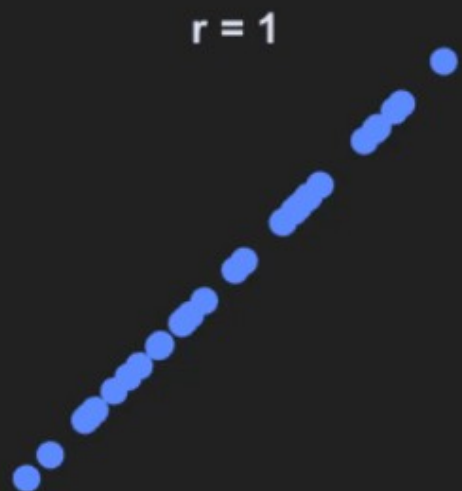
Zero correlation



Zero correlation

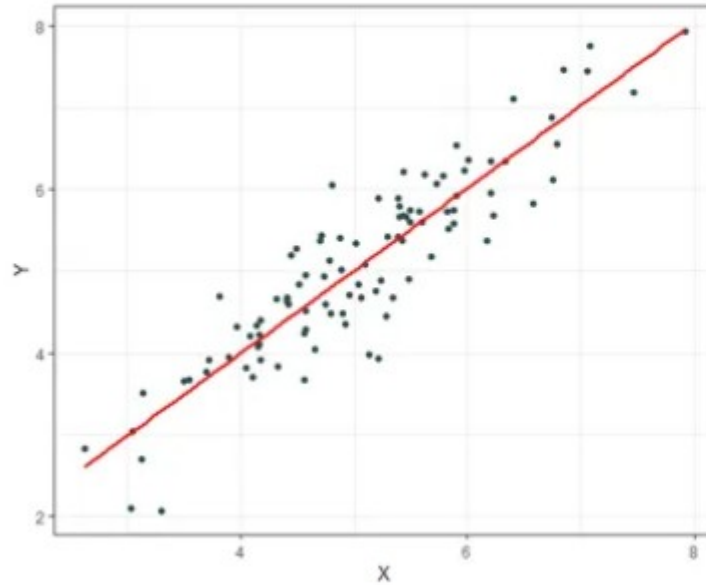


Correlation strengths

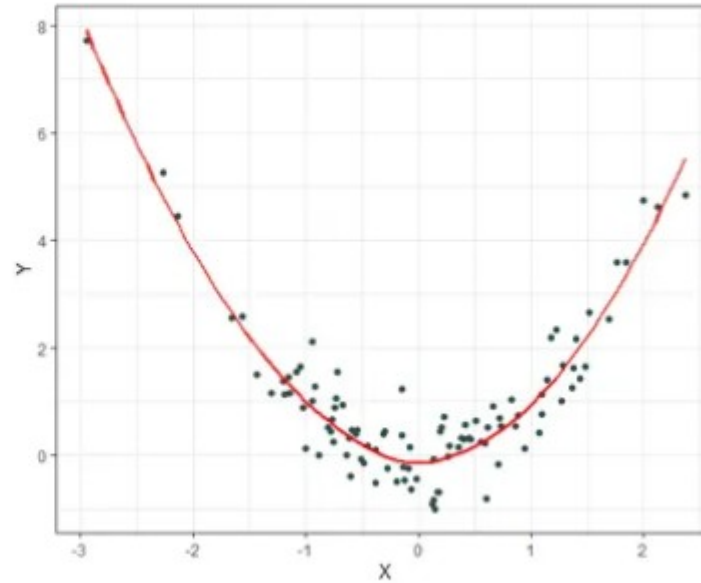


A New Coefficient of Correlation : Tim Sumner (Medium)

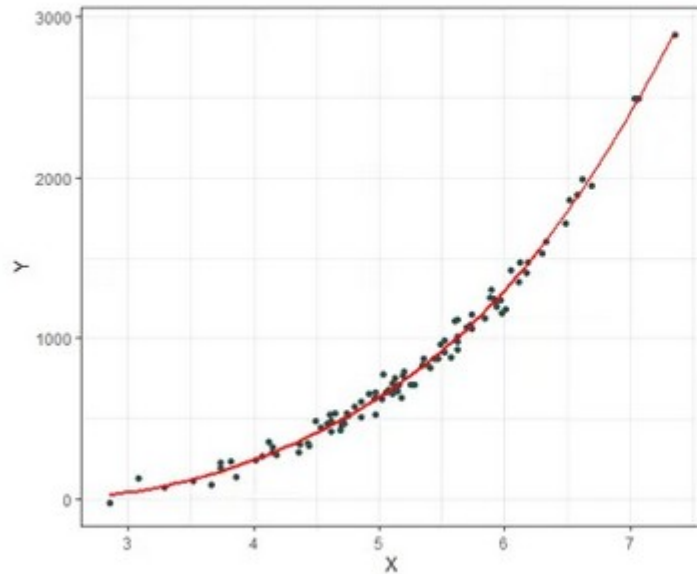
Linear



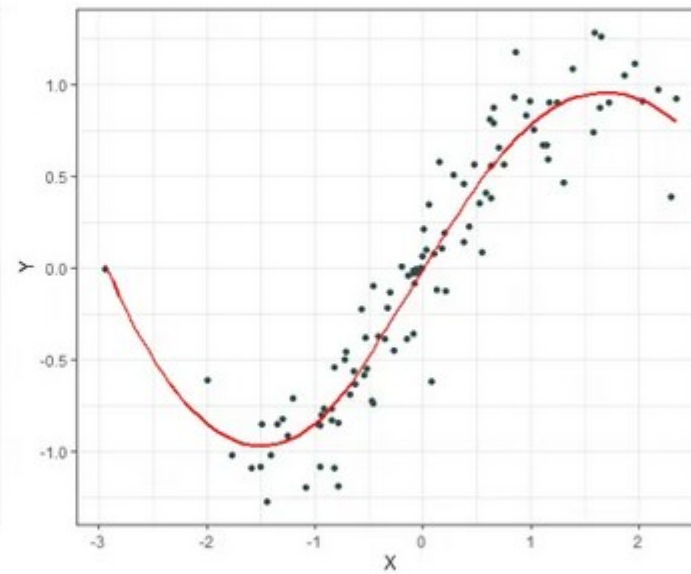
Non-Linear



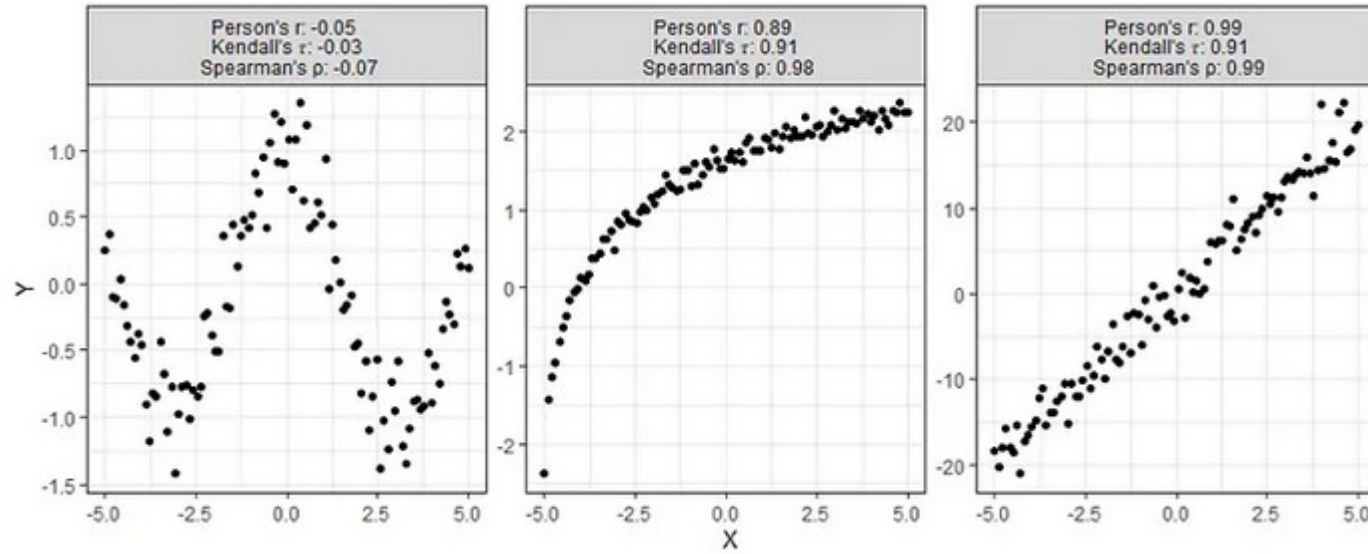
Monotonic



Non-Monotonic



It should come as no surprise to most that it is rare to observe linear relationships in the real world. This is why other measures have been created over the decades such as Spearman's ρ (rho) and Kendall's τ (tau) to name a few. These newer measures are much better at identifying monotonic relationships and not just linear ones which makes them more robust since linear relationships are a specific type of monotonic relationship.



Despite having obvious shortcomings, these correlations are still being used to make many conclusions about all sorts of data. Is there any hope for identifying relationships that are even more complex than the ones shown above? Enter, the new coefficient ξ (pronounced “ksaai”). One last note before moving on, a paper was published in 2022 titled [Myths About Linear and Monotonic Associations](#)[2] related to the issue of stating which popular measure of correlation is preferred for which type of data.

[1] S. Chatterjee, [A New Coefficient of Correlation](#) (2020), Journal of the American Statistical Association.

[2] E. Heuvel and Z. Zhan, [Myths About Linear and Monotonic Associations](#) (2022), The American Statistician.

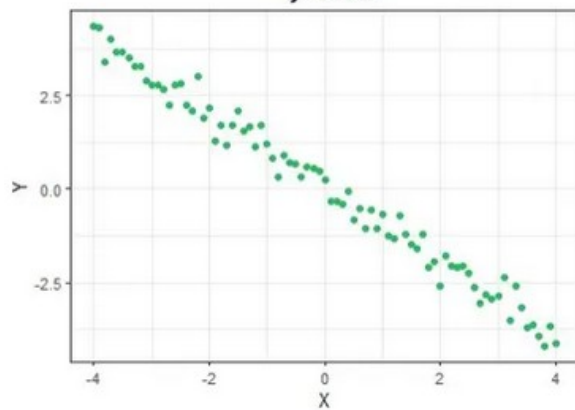
There are two formulas used depending on the type of data you are working with. If ties in your data are impossible (or extremely unlikely), we have

$$\xi_n(X, Y) := 1 - \frac{3 \sum_{i=1}^{n-1} |r_{i+1} - r_i|}{n^2 - 1}$$

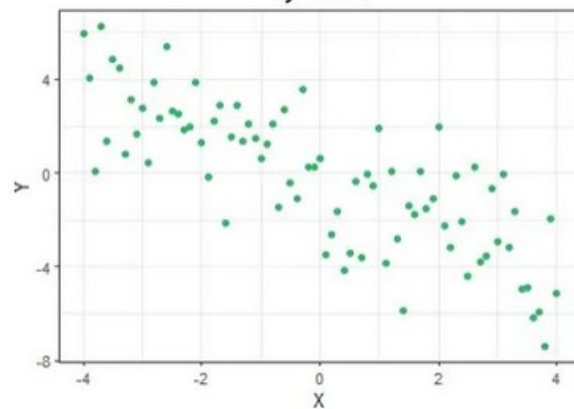
and if ties are allowed, we have

$$\xi_n(X, Y) := 1 - \frac{n \sum_{i=1}^{n-1} |r_{i+1} - r_i|}{2 \sum_{i=1}^n l_i (n - l_i)}$$

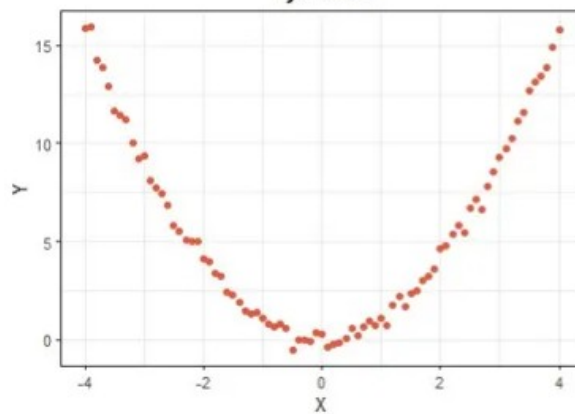
Pearson's r: -0.99
 ξ : 0.85



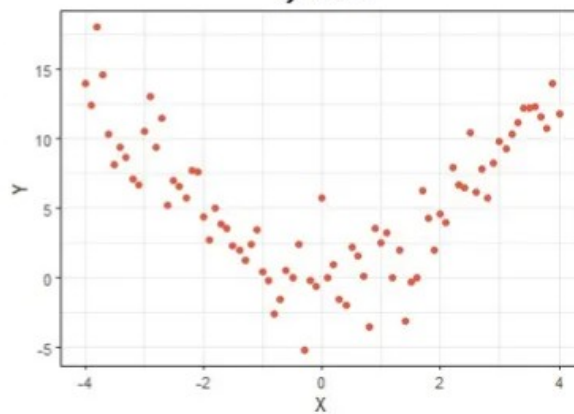
Pearson's r: -0.78
 ξ : 0.34



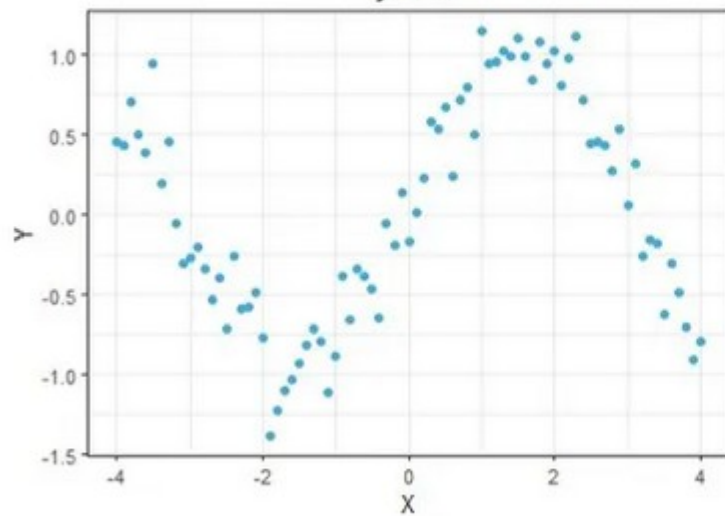
Pearson's r: 0.01
 ξ : 0.89



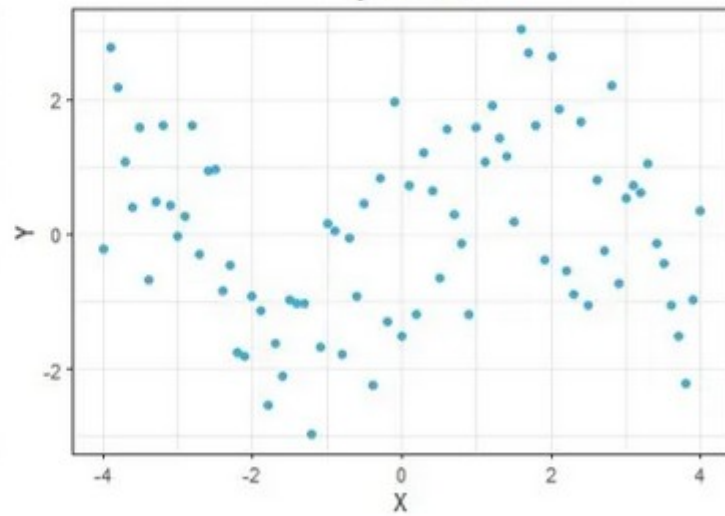
Pearson's r: -0.03
 ξ : 0.64



Pearson's r: 0.25
 ξ : 0.7



Pearson's r: 0.05
 ξ : 0.18



Correlation vs. causation

Correlation merely shows a relationship.

It does not reveal or imply causality.

Causality can be demonstrated by experimental manipulations.

Example: Ice cream $\Rightarrow$ shark attacks



Covariance and correlation: formulas

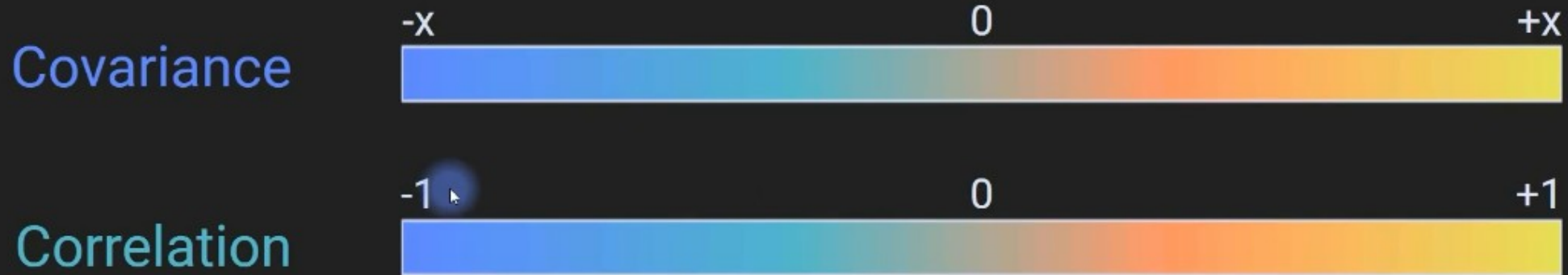
In this video, you will learn

- The difference between covariance and correlation.
- The formulas.

Covariance vs. correlation: what's the diff?

Covariance is a single number that measures the linear relationship between two variables.

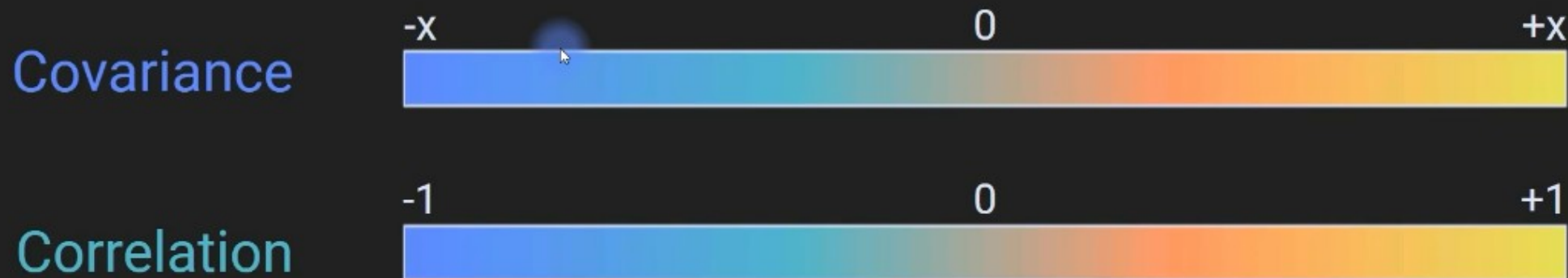
Correlation is the scaled covariance.



Covariance vs. correlation: what's the diff?

Covariance is in the same scale as the original data. It is not normalized.

Correlation is normalized to be independent of the data scale.



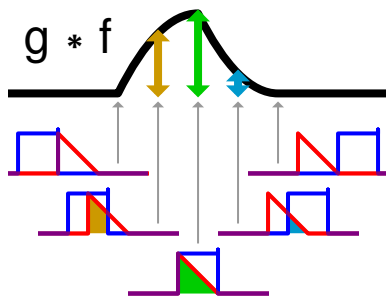
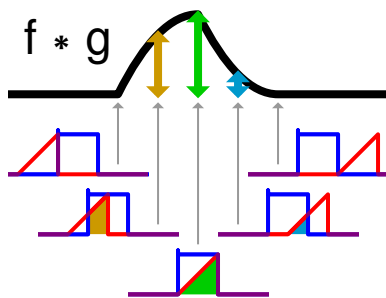
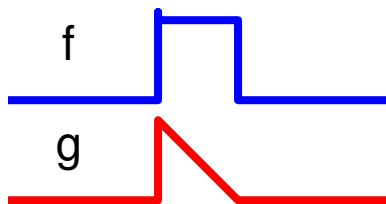
Formula for covariance and correlation

$$c = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

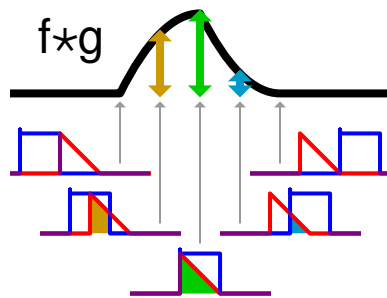
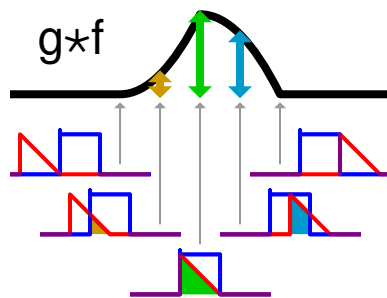
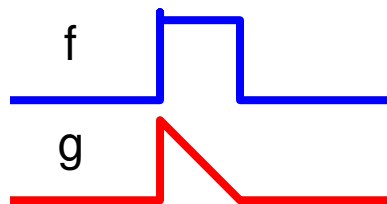
$$\text{Correlation} = \frac{\text{Cov}(x, y)}{\sigma_x * \sigma_y}$$

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

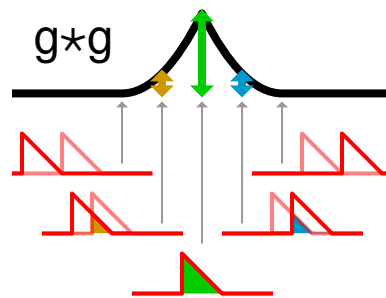
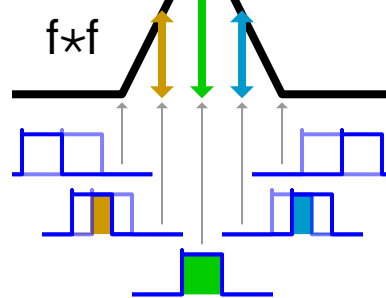
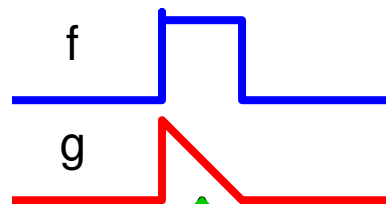
Convolution

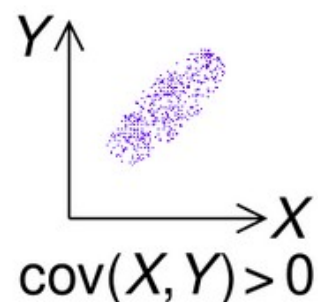
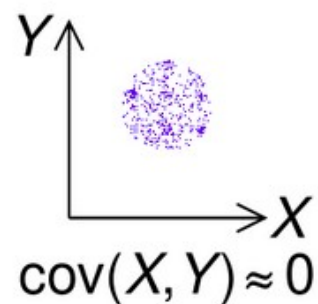
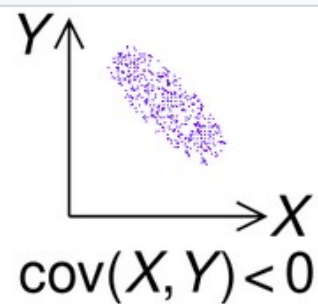
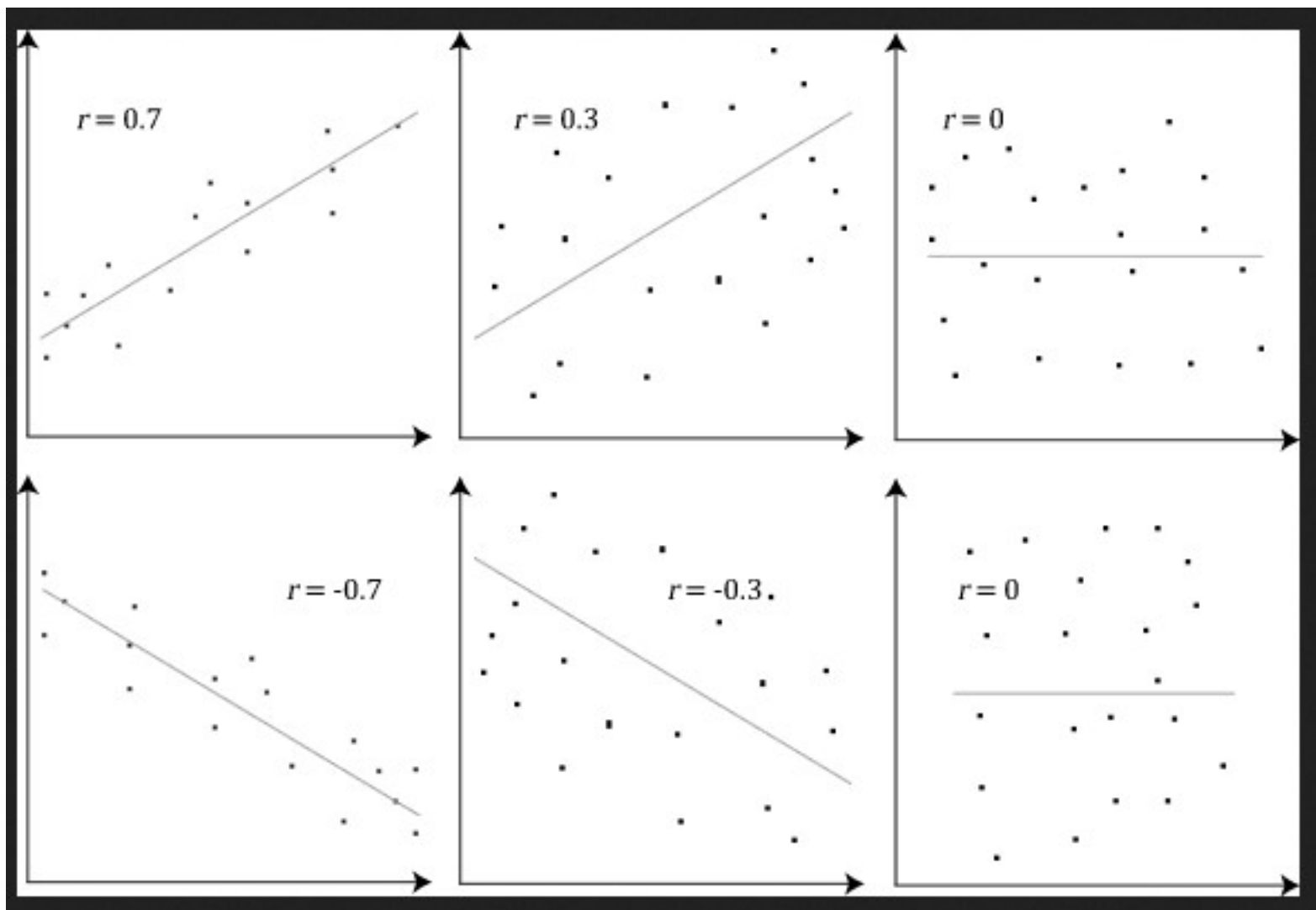


Cross-correlation



Autocorrelation





The sign of the covariance of two random variables X and Y

P-value of correlation coefficient

$$t_{n-2} = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}}$$

Statistical significance is computed from a t-value that is based on the strength of the correlation and the number of data points.

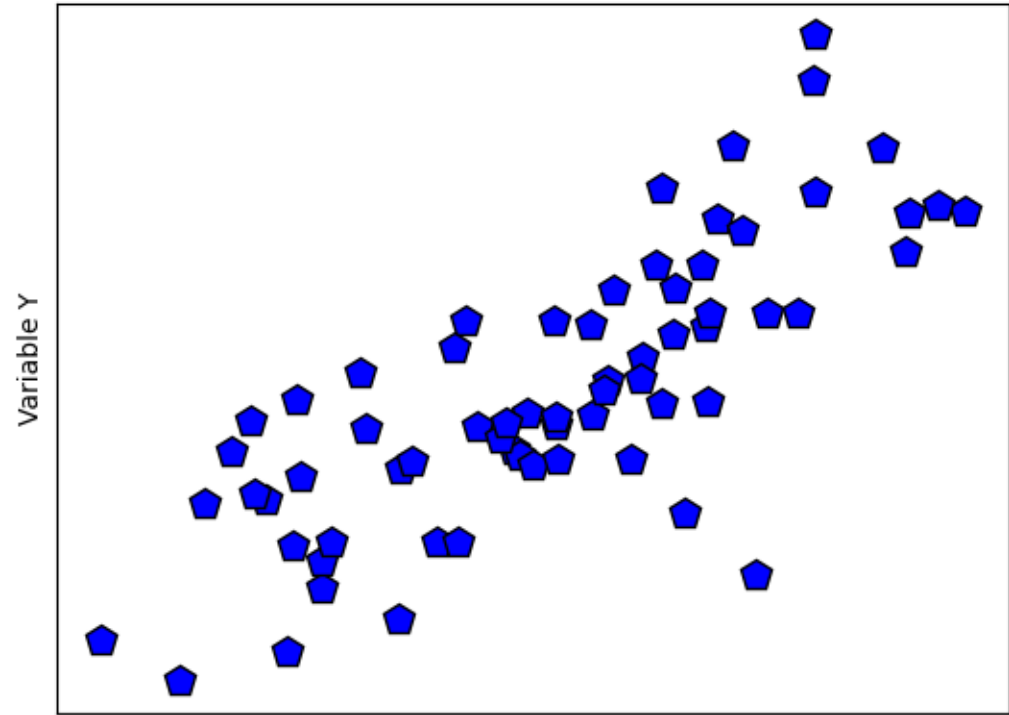
```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## simulate data
N = 66
# generate correlated data
x = np.random.randn(N)
y = x + np.random.randn(N)
# plot the data
plt.plot(x,y,'kp',markerfacecolor='b',markersize=12)
plt.xlabel('Variable X')
plt.ylabel('Variable Y')
plt.xticks([])
plt.yticks([])
plt.show()

## compute covariance
# precompute the means
meanX = np.mean(x)
meanY = np.mean(y)
### the loop method
covar1 = 0
for i in range(N):
    covar1 = covar1 + (x[i]-meanX)*(y[i]-meanY)
# and now for the normalization
covar1 = covar1/(N-1)
### the linear algebra method
xCent = x-meanX
yCent = y-meanY
covar2 = np.dot(xCent,yCent) / (N-1)
### the Python method
covar3 = np.cov(np.vstack((x,y)))
print(covar1,covar2,covar3)

```



covar1

covar2

covar3

```

warnings.warn('Unable to import Axes3D. This may be due to multiple versions of
1.1106800901103804 1.1106800901103804 [[1.11639168 1.11068009]
[1.11068009 1.84721683]]
0.7734212150041268 [[1.11068009 1.84721683]

```

```
## now for correlation
### the long method
corr_num = sum( (x-meanX) * (y-meanY) )
corr_den = sum((x-meanX)**2) * sum((y-meanY)**2)
corr1 = corr_num/np.sqrt(corr_den)
### the Python method
corr2 = np.corrcoef(np.vstack((x,y)))
print(corr1,corr2)
```

```
## correlation as normalized covariance
xn = stats.zscore(x,ddof=1)
yn = stats.zscore(y)
corr3 = np.dot(xn,yn) / (N-1)
print(corr3)
```

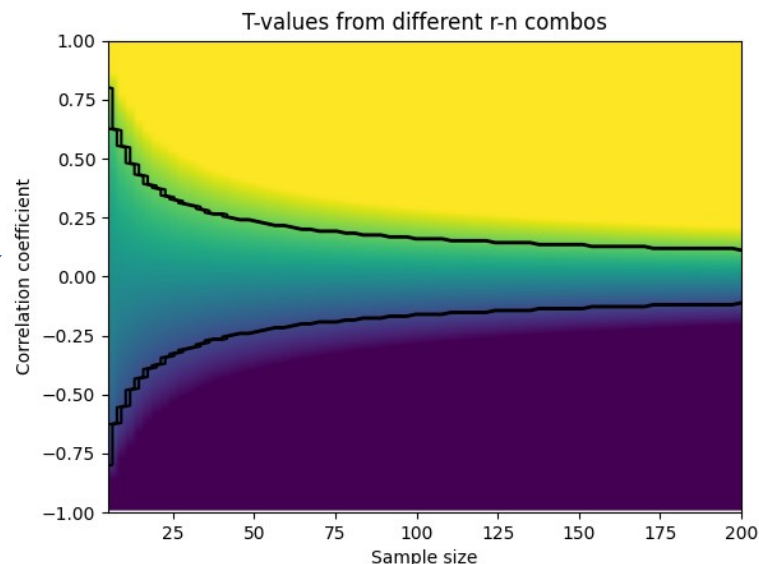
```
## 2D t-value space based on r and n
# define parameters
r = np.linspace(-1,1,248)
n = np.round( np.linspace(5,200,73) )
# initialize t-value matrix
tmatrix = np.zeros((len(r),len(n)))
pmatrix = np.zeros((len(r),len(n)))
for ri in range(len(r)):
    for ni in range(len(n)):
        # the t-value, split into num/den
        num = r[ri]*np.sqrt(n[ni]-2)
        den = np.sqrt(1-r[ri]**2)
        tmatrix[ri,ni] = num/den
        pmatrix[ri,ni] = 1-stats.t.cdf(abs(num/den),n[ni]-2)
# Soooo curious to see it!
plt.imshow(tmatrix,vmin=-3,vmax=3,extent=[n[0],n[-1],r[0],r[-1]],aspect='auto',origin='lower')
plt.contour(pmatrix<.05,1,colors='k',extent=[n[0],n[-1],r[0],r[-1]])
plt.xlabel('Sample size')
plt.ylabel('Correlation coefficient')
plt.title('T-values from different r-n combos')
plt.show()
```

```
# question: Why the warning message?
# final note on statistical significance
```

```
r,p = stats.pearsonr(x,y)
print(r,p)
```

```
[[1.11000005 1.04721005]]
0.7734313158844368 [[1.          0.77343132]
[0.77343132 1.          ]]
0.7793580793071677
```

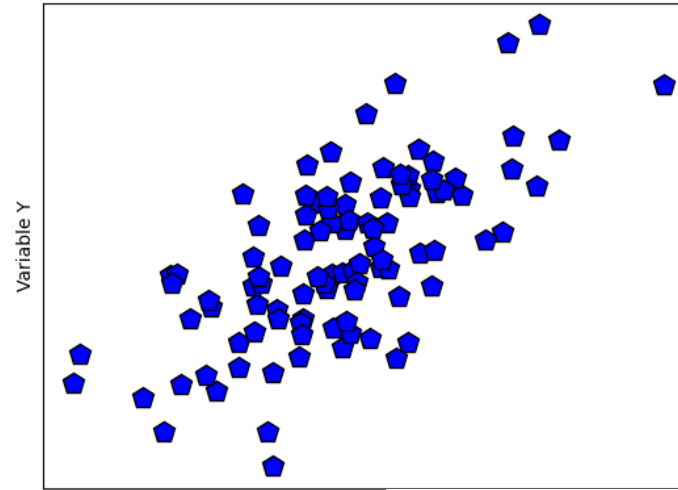
```
0.7793580793071677
/home/atalet/workspace
```



```
tmatrix[ri,ni] = num/den
/home/atalet/workspace/YZ_EGITIM/ozetler/MATH-02-Master
pmatrix[ri,ni] = 1-stats.t.cdf(abs(num/den),n[ni]-2)
0.7734313158844369 2.709392421109592e-14
```

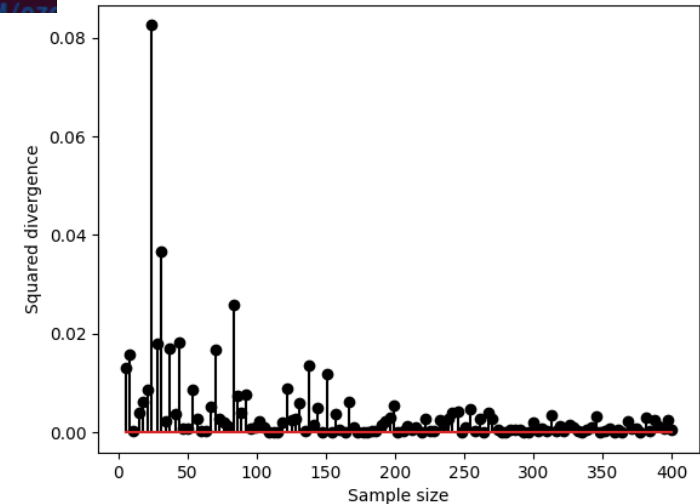
SIMULATE DATA WITH SPECIFIED CORRELATION

```
import matplotlib.pyplot as plt
import numpy as np
## simulate data
# data simulation parameters
N = 100 # number of samples
r = .6 # desired correlation coefficient
# start with random numbers
x = np.random.randn(N)
y = np.random.randn(N)
# impose the correlation on y
y = x*r + y*np.sqrt(1-r**2)
# plot the data
plt.plot(x,y,'kp',markerfacecolor='b',markersize=12)
plt.xlabel('Variable X')
plt.ylabel('Variable Y')
plt.xticks([])
plt.yticks([])
plt.show()
```



```
## compute the empirical correlation
empR = np.corrcoef(x,y)
print('Desired r=%g, empirical r=%g'%(r,empR[0,1]))
## Test the errors as a function of N
# range of sample sizes
Ns = np.round( np.linspace(5,400,123) ).astype(int)
# theoretical correlation coefficient (fixed)
r = .6
# initialize
corrs = np.zeros(len(Ns))
# run the experiment!
for ni in range(len(Ns)):
    x = np.random.randn(Ns[ni])
    y = x*r + np.random.randn(Ns[ni])*np.sqrt(1-r**2)
    corrs[ni] = (r-np.corrcoef(x,y)[0,1])**2
plt.stem(Ns,corrs,'ko-')
plt.xlabel('Sample size')
plt.ylabel('Squared divergence')
plt.show()
```

```
warnings.warn('Unable to import Axes',
Desired r=0.6, empirical r=0.676427
*alest@uzges: /workspaces/V3_50TTM/...
```

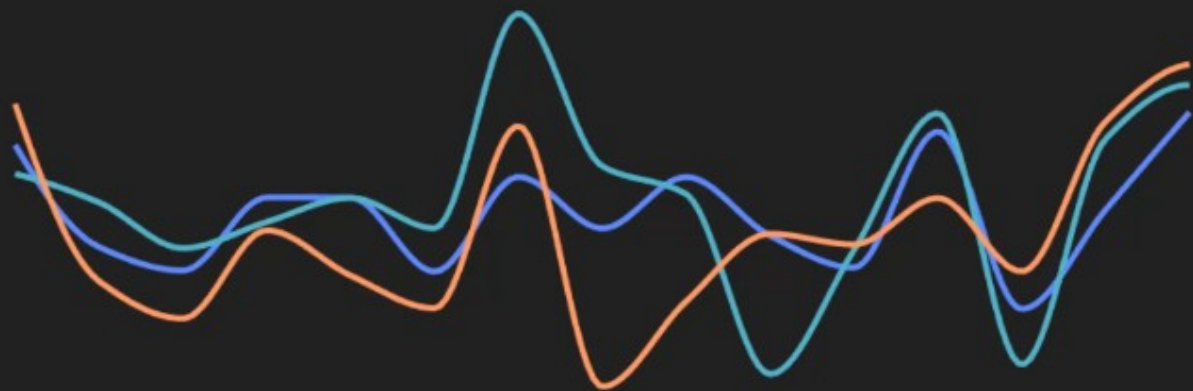


Correlation matrix

In this video, you will learn

- How to organize correlations across multiple variables.
- Why correlation matrices (and correlations more generally) are symmetric.

Correlation matrix



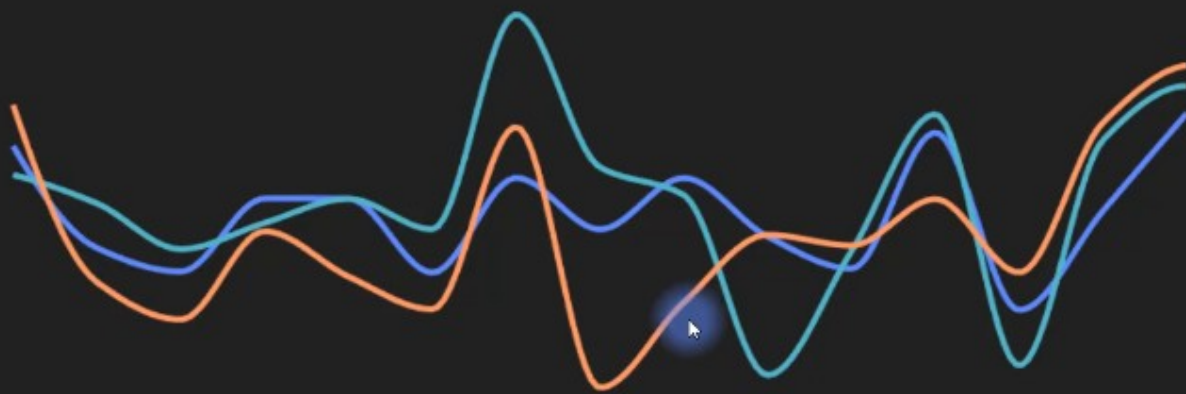
	1	2	3
1	1	.67	.51
2	.67	1	.41
3	.51	.41	1

Formula for covariance and correlation

$$c = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

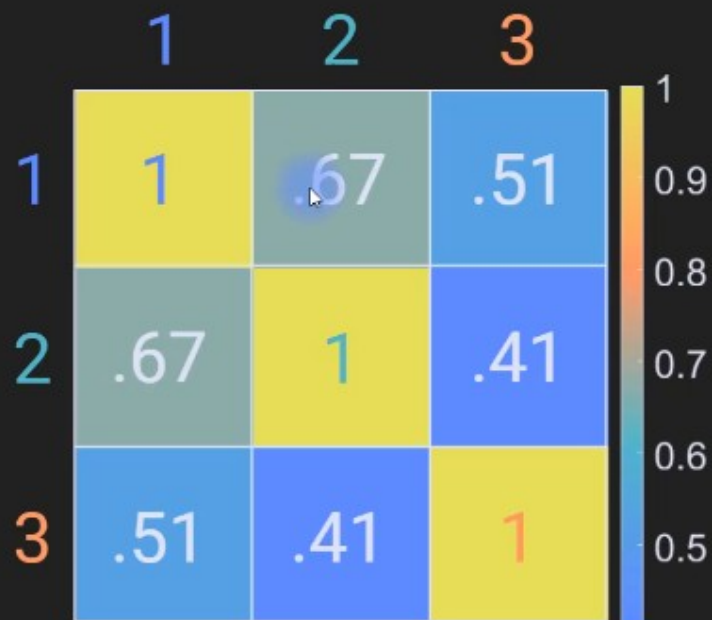
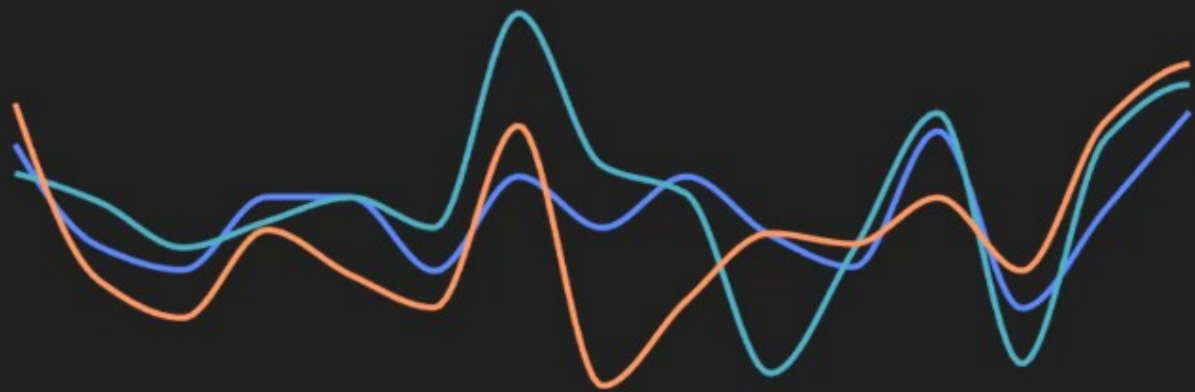
$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Covariance matrix

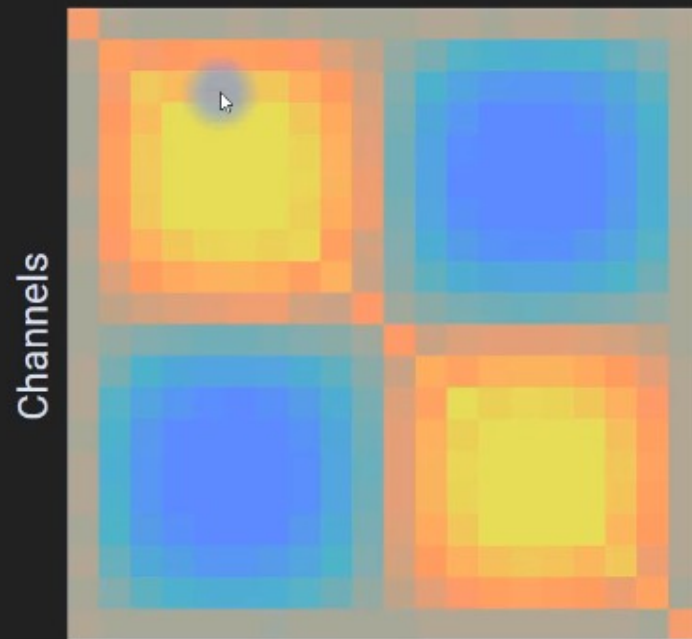
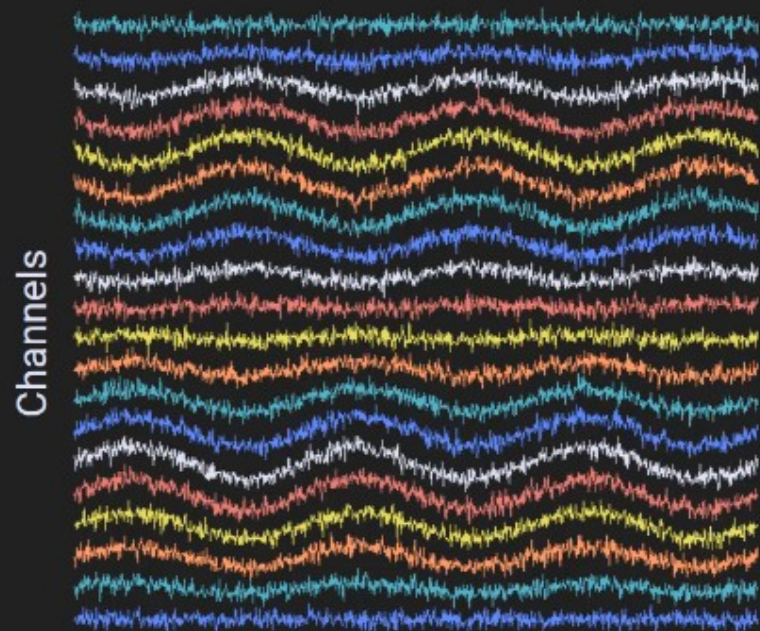


	1	2	3
1	3.16	4.13	2.79
2	4.13	11.79	4.32
3	2.79	4.32	9.31

Visualizing correlation matrices

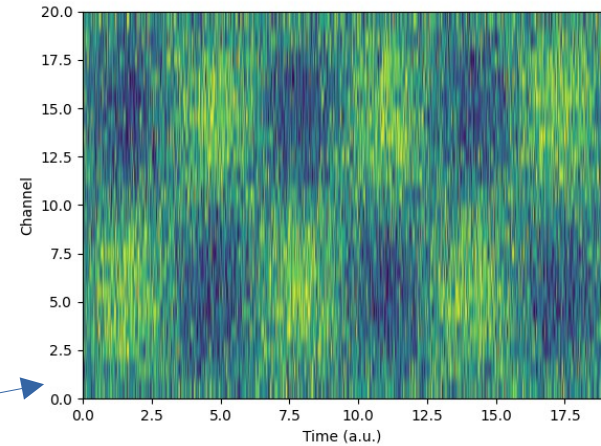
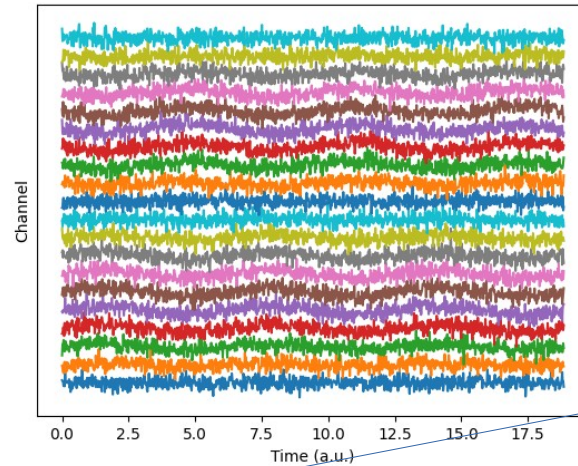


Visualizing correlation matrices



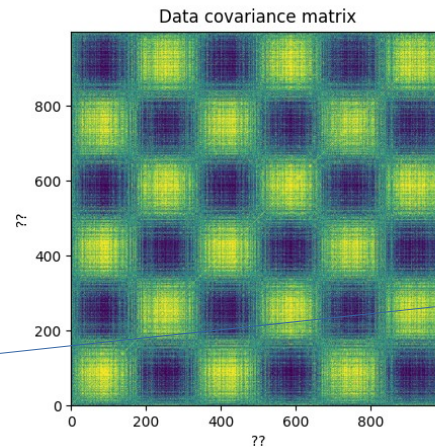
```
import matplotlib.pyplot as plt
import numpy as np
import numpy.matlib as matlib

## simulate data
# simulation parameters
N = 1000 # time points
M = 20 # channels
# time vector (radian units)
t = np.linspace(0,6*np.pi,N)
# relationship across channels (imposing covariance)
chanrel = np.sin(np.linspace(0,2*np.pi,M))
# create the data
data = np.zeros((M,N))
for mi in range(M):
    data[mi,:] = np.sin(t) * chanrel[mi]
data = data + np.random.randn(M,N)
# two ways of visualizing the multichannel data
for i in range(M):
    plt.plot(t,data[i,:]+i*4)
```

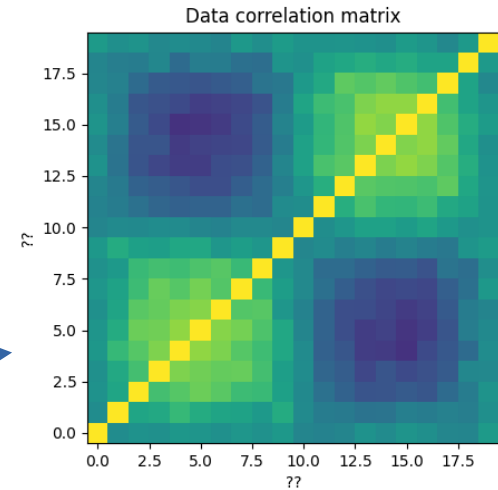


```
plt.yticks([])
plt.xlabel('Time (a.u.)')
plt.ylabel('Channel')
plt.show()
plt.imshow(data,aspect='auto',vmin=-2,vmax=2,extent=[t[0],t[-1],0,M],origin='lower')
plt.xlabel('Time (a.u.)')
plt.ylabel('Channel')
plt.show()
```

```
## now compute the covariance matrix
# note the size of the output!
dataCovMat = np.cov(data.T)
plt.imshow(dataCovMat,vmin=-.5,vmax=.5,origin='lower')
plt.title('Data covariance matrix')
plt.xlabel('??')
plt.ylabel('??')
plt.show()
```



```
## and now the correlation matrix
# note the size of the output!
dataCorrMat = np.corrcoef(data)
plt.imshow(dataCorrMat,vmin=-.5,vmax=.5,origin='lower')
plt.title('Data correlation matrix')
plt.xlabel('??')
plt.ylabel('??')
plt.show()
```



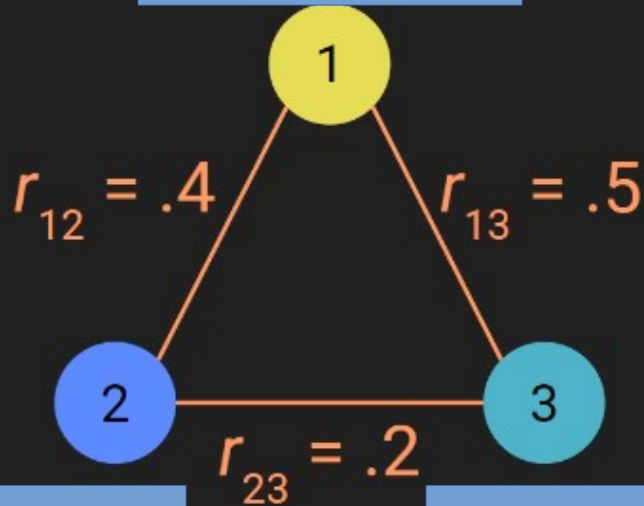
Partial correlation

In this video, you will learn

- Why correlations among multiple variables might need to be “partialled.”
- The formula for the partial correlation.

Partial correlation: motivation and effect

Good weather

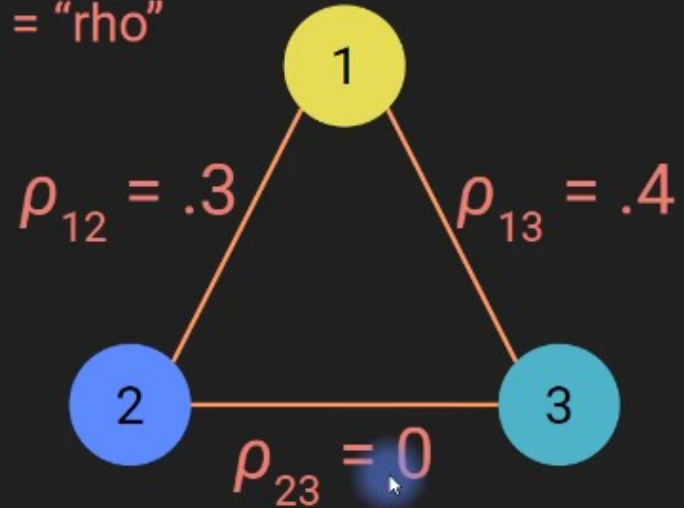


Ice cream consumption

Shark attacks



ρ = "rho"



Partial correlation: example application

What is the correlation between socioeconomic status (m) and GMAT (standardized business school exam) (g), when partialling out hours spent studying the GMAT (s)?

$$r_{mg} = .7$$

$$r_{sg} = .8$$

$$r_{ms} = .9$$



$$\rho_{mg|s} = .0$$

$$\rho_{sg|m} = .5$$

$$\rho_{ms|g} = \text{NA}$$

Partial correlation: formula

$$\rho_{xy|z} = \frac{r_{xy} - r_{xz}r_{yz}}{\sqrt{1 - r_{xz}^2} \sqrt{1 - r_{yz}^2}}$$

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy.stats as stats

# I'm using pingouin for partial correlations.
# You might need to install it, e.g., using the line below.
# This needs to be run only once per install.
# conda install -c conda-forge pingouin
import pingouin as pg
## the example from the video
# raw correlations
rmg = .7
rsg = .8
rms = .9

# partial correlations
rho_mg_s = (rmg - rsg*rms) / ( np.sqrt(1-rsg**2)*np.sqrt(1-rms**2) )
rho_sg_m = (rsg - rmg*rms) / ( np.sqrt(1-rmg**2)*np.sqrt(1-rms**2) )
print(rho_mg_s)
print(rho_sg_m)

## now for datasets
N = 76
# correlated datasets
x1 = np.linspace(1,10,N) + np.random.randn(N)
x2 = x1 + np.random.randn(N)
x3 = x1 + np.random.randn(N)
# let's convert these data to a pandas frame
df = pd.DataFrame()
df['x1'] = x1
df['x2'] = x2
df['x3'] = x3
# compute the "raw" correlation matrix
cormatR = df.corr()
print(cormatR)
# print out one value
print(' ')
print(cormatR.values[1,0])
# partial correlation
pc = pg.partial_corr(df, x='x3', y='x2', covar='x1')
print(' ')
print(pc)

```

```

from pandas.core import i
-0.07647191129018778
0.5461186812727504
      x1      x2      x3
x1  1.000000  0.926511  0.927786
x2  0.926511  1.000000  0.875150
x3  0.927786  0.875150  1.000000

0.9265106283423946

      n      r      CI95      p_val
pearson  76  0.110735  [-0.12, 0.33]  0.344251

```

```

## visualize the matrices
fig, ax = plt.subplots(1, 2, figsize=(6, 3))

# raw correlations
ax[0].imshow(cormatR.values, vmin=-1, vmax=1)
ax[0].set_xticks(range(3))
ax[0].set_yticks(range(3))

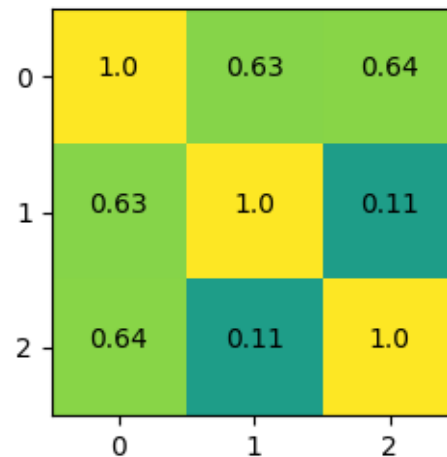
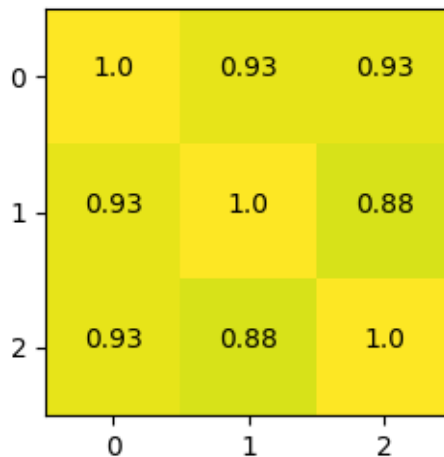
# add text
for i in range(3):
    for j in range(3):
        ax[0].text(i, j, np.round(cormatR.values[i, j], 2), horizontalalignment='center')

# partial correlations
partialCorMat = df.pcorr()
ax[1].imshow(partialCorMat.values, vmin=-1, vmax=1)
ax[1].set_xticks(range(3))
ax[1].set_yticks(range(3))

for i in range(3):
    for j in range(3):
        ax[1].text(i, j, np.round(partialCorMat.values[i, j], 2), horizontalalignment='center')

plt.show()

```



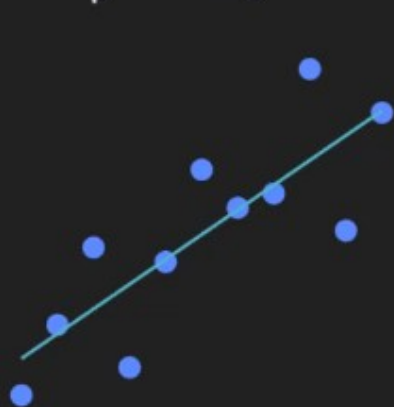
The problem with Pearson

In this video, you will learn

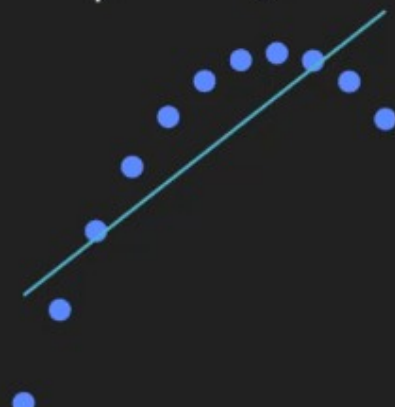
- The limitations of the Pearson correlation coefficient.

Anscombe's quartet

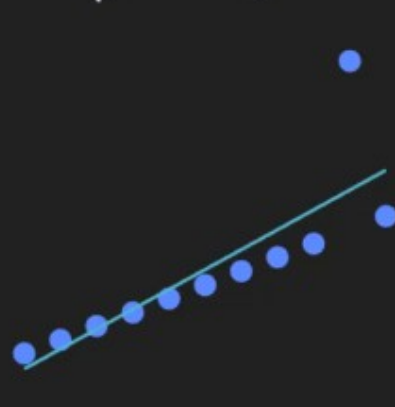
$$r_p = 0.82; r_s = 0.82$$



$$r_p = 0.82; r_s = 0.69$$



$$r_p = 0.82; r_s = 0.99$$



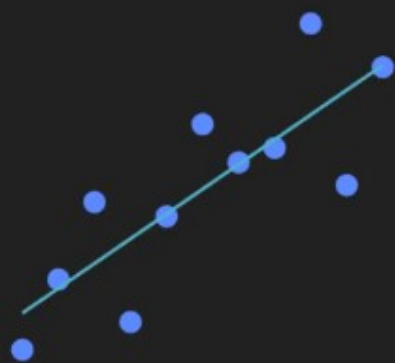
$$r_p = 0.82; r_s = 0.5$$



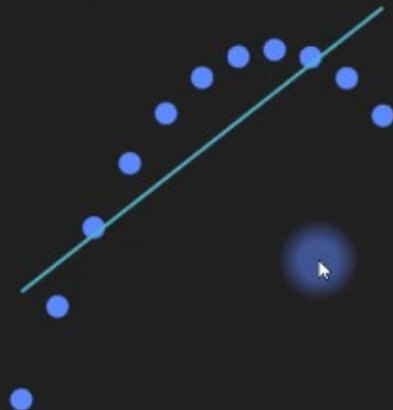
Conclusion: Pearson correlation can over- or under-represent relationships if they contain nonlinearities or outliers. Pearson is appropriate for normally distributed data.

Anscombe's quartet

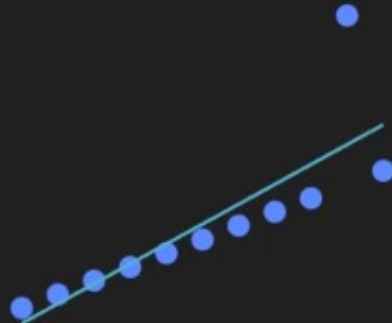
$r_p = 0.82$; $r_s = 0.82$



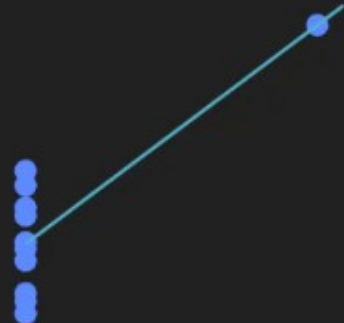
$r_p = 0.82$; $r_s = 0.69$



$r_p = 0.82$; $r_s = 0.99$



$r_p = 0.82$; $r_s = 0.5$



Conclusions:

Pearson's r inflates or deflates nonlinear relationships.

Pearson's r is sensitive to outliers.

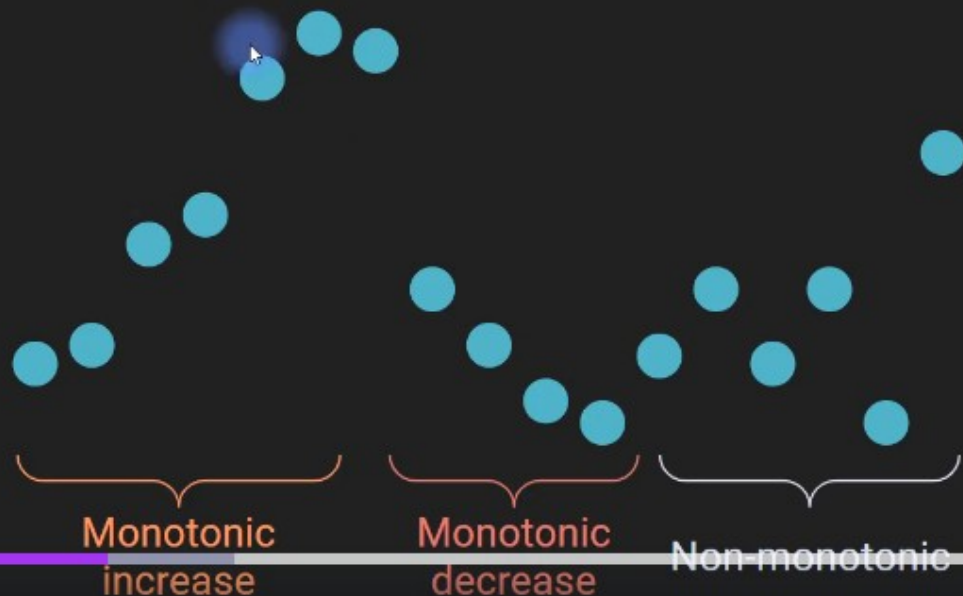
Thus, Pearson's r is appropriate for normal data without outliers.

Pearson and Spearman converge when data are normally distributed.

Spearman (rank) correlation

Spearman's rho tests for a monotonic relationship, regardless of whether the relationship is linear or nonlinear.

A monotonic relationship tests for increasing or decreasing numbers, regardless of the spacing between numbers.



How to compute the Spearman correlation

Step 1: Transform both variables to rank.

([3321654, -40, 1, 0] $\Rightarrow$ [4, 1, 3, 2]).

Step 2: Compute Pearson correlation coefficient on ranks.

P-value: Same as for “regular” Pearson coefficient.

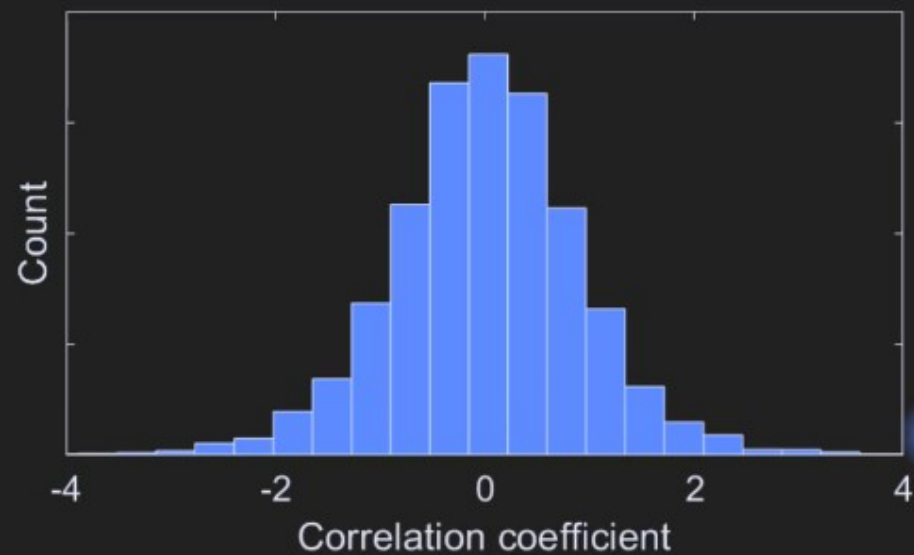
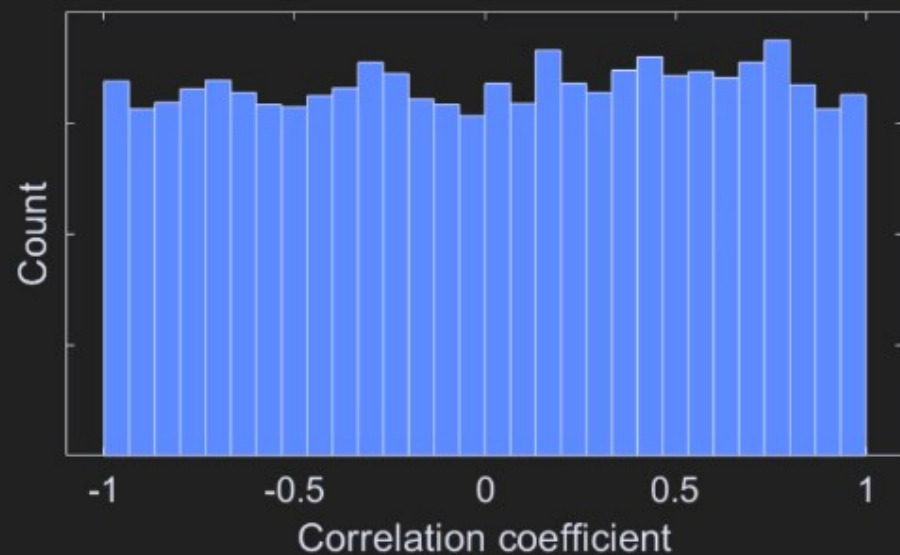
$$t_{n-2} = \frac{r\sqrt{n-2}}{1-r^2}$$

Fisher-Z transform for correlations

In this video, you will learn

- Why correlations sometimes need to be transformed.
- The full formula for the Fisher-Z transformation.
- The equivalent shortcut.

Why do correlations need a transformation?

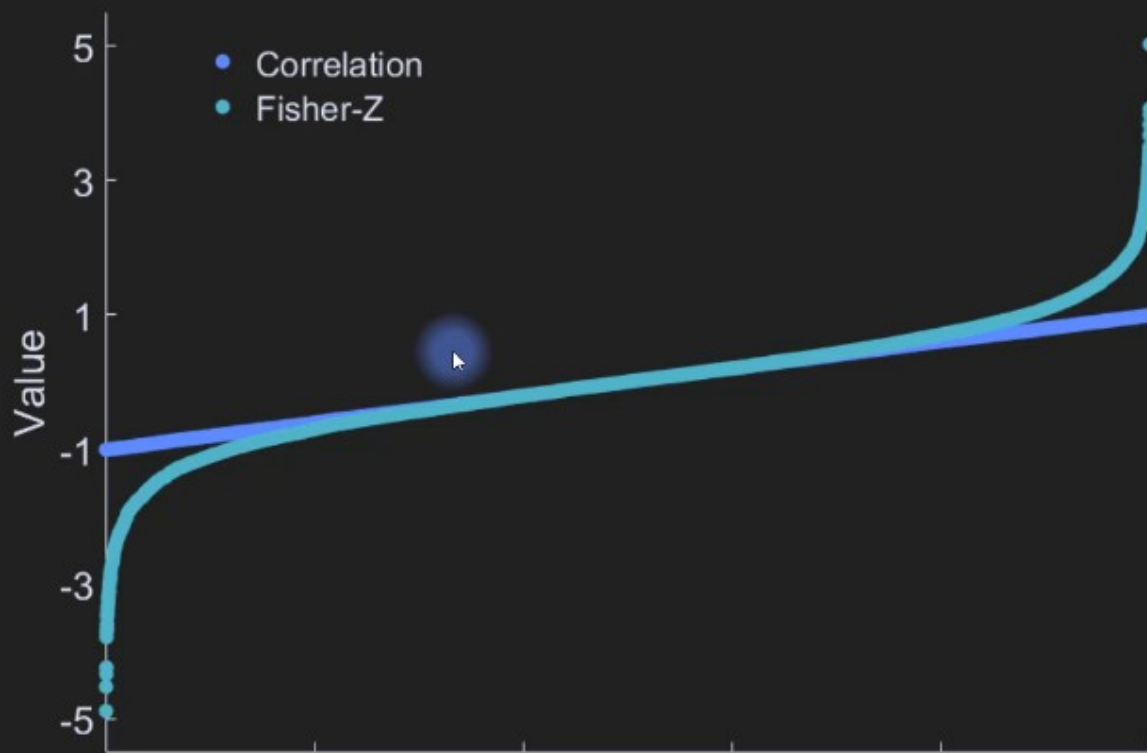


Correlation coefficients are uniformly distributed -1 to +1.
Many analysis methods assume normal distributions.

How does the Fisher-Z transform work?

$$z_r = \frac{1}{2} \ln \left(\frac{1+r}{1-r} \right) = \operatorname{arctanh}(r)$$

The effect of the Fisher-z transform



```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## Anscombe's quartet
anscombe = np.array([
    # series 1      series 2      series 3      series 4
    [10, 8.04, 10, 9.14, 10, 7.46, 8, 6.58, ],
    [ 8, 6.95,  8, 8.14,  8, 6.77,  8, 5.76, ],
    [13, 7.58, 13, 8.76, 13, 12.74, 8, 7.71, ],
    [ 9, 8.81,  9, 8.77,  9, 7.11,  8, 8.84, ],
    [11, 8.33, 11, 9.26, 11, 7.81,  8, 8.47, ],
    [14, 9.96, 14, 8.10, 14, 8.84,  8, 7.04, ],
    [ 6, 7.24,  6, 6.13,  6, 6.08,  8, 5.25, ],
    [ 4, 4.26,  4, 3.10,  4, 5.39,  8, 5.56, ],
    [12, 10.84, 12, 9.13, 12, 8.15,  8, 7.91, ],
    [ 7, 4.82,  7, 7.26,  7, 6.42,  8, 6.89, ],
    [ 5, 5.68,  5, 4.74,  5, 5.73, 19, 12.50, ]
])

# plot and compute correlations
fig, ax = plt.subplots(2, 2, figsize=(6, 6))
ax = ax.ravel()

for i in range(4):
    ax[i].plot(anscombe[:, i*2], anscombe[:, i*2+1], 'ko')
    ax[i].set_xticks([])
    ax[i].set_yticks([])
    corr_p = stats.pearsonr(anscombe[:, i*2], anscombe[:, i*2+1])[0]
    corr_s = stats.spearmanr(anscombe[:, i*2], anscombe[:, i*2+1])[0]
    ax[i].set_title('r_p = %g, r_s = %g'%(np.round(corr_p*100)/100, np.round(corr_s*100)/100))

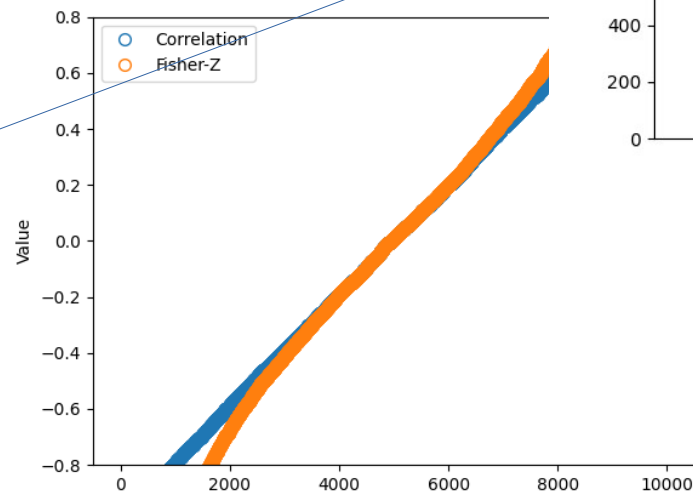
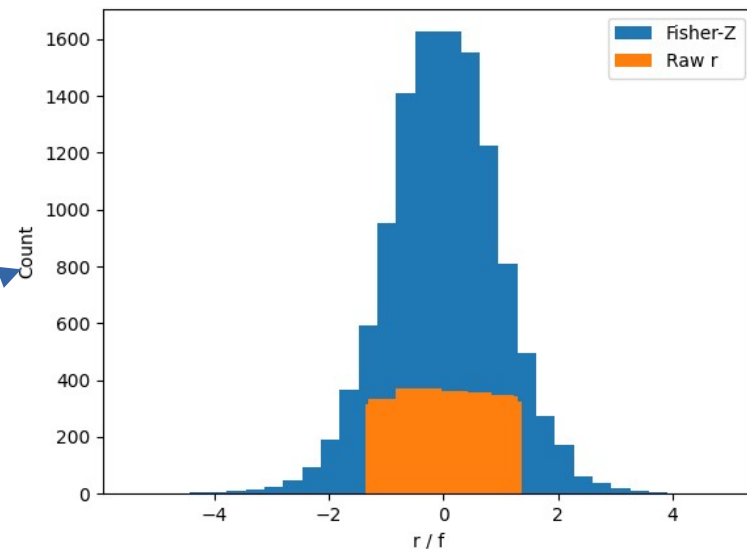
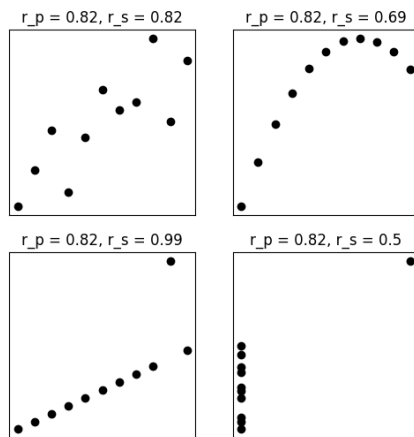
plt.show()

## Fisher-Z transform
# simulate correlation coefficients
N = 10000
r = 2*np.random.rand(N) - 1
# Fisher-Z
fz = np.arctanh(r)
# overlay the Fisher-Z
y, x = np.histogram(fz, 30)
x = (x[1:] + x[0:-1]) / 2
plt.bar(x, y)
# raw correlations
y, x = np.histogram(r, 30)
x = (x[1:] + x[0:-1]) / 2
plt.bar(x, y)
plt.xlabel('r / f')
plt.ylabel('Count')
plt.legend(('Fisher-Z', 'Raw r'))
plt.show()

plt.plot(range(N), np.sort(r), 'o', markerfacecolor='w', markersize=7)
plt.plot(range(N), np.sort(fz), 'o', markerfacecolor='w', markersize=7)
plt.ylabel('Value')
plt.legend(('Correlation', 'Fisher-Z'))

# zoom in
plt.ylim([-0.8, 0.8])
plt.show()

```



Kendall's correlation

In this video, you will learn

- When the Kendall correlation is appropriate.
- The basic formula for Kendall's correlation.

Kendall's correlation

Used for ordinal data (numerical meaningful order, but no fixed relationship across the levels).

Examples: education, movie ratings

★★★★★	Loved it
★★★★☆	Liked it
★★★☆☆	It was ok
★★★☆☆	Disliked it
★☆☆☆☆	Hated it

How it works

Transform the data to rank concordances (relative signs across the values per variable).

(See next slide for overview of formula.)

“Kendall tau-b” has an adjustment for ties, and is most often used.

Interpretation is identical to Pearson and Spearman.

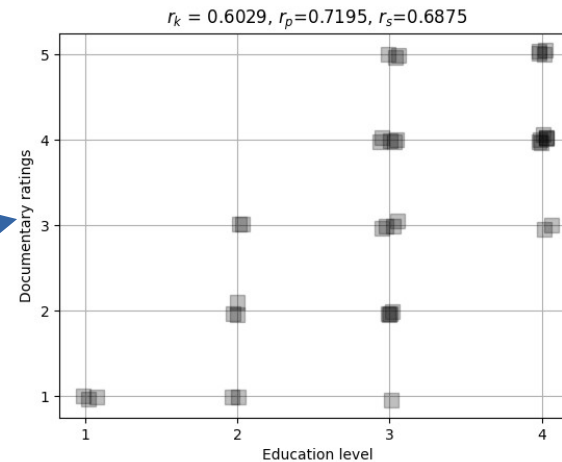
How it works

Transform the data to rank concordances (relative signs across the values per variable).

$$\tau = K^{-1} \sum \text{sgn}(\tilde{x}_i - \tilde{x}_{i:}) \text{sgn}(\tilde{y}_i - \tilde{y}_{i:})$$

```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## generate some data!
N = 40
# movie ratings
docuRatings = np.random.randint(low=1,high=6,size=N)
# education level (1-4, correlated with docuRatings)
eduLevel = np.ceil( (docuRatings + np.random.randint(low=1,high=5,size=N) )/9 * 4 )
# compute the correlations
cr = [0,0,0]
cr[0] = stats.kendalltau(eduLevel,docuRatings)[0]
cr[1] = stats.pearsonr(eduLevel,docuRatings)[0]
cr[2] = stats.spearmanr(eduLevel,docuRatings)[0]
# round for convenience
cr = np.round(cr,4)
# plot the data
plt.plot(eduLevel+np.random.randn(N)/30,docuRatings+np.random.randn(N)/30,'ks',markersize=10,markerfacecolor=[0,0,0,.25])
plt.xticks(np.arange(4)+1)
plt.yticks(np.arange(5)+1)
plt.xlabel('Education level')
plt.ylabel('Documentary ratings')
plt.grid()
plt.title('$r_k$ = %g, $r_p$=%g, $r_s$=%g'%(cr[0],cr[1],cr[2]))
plt.show()
```



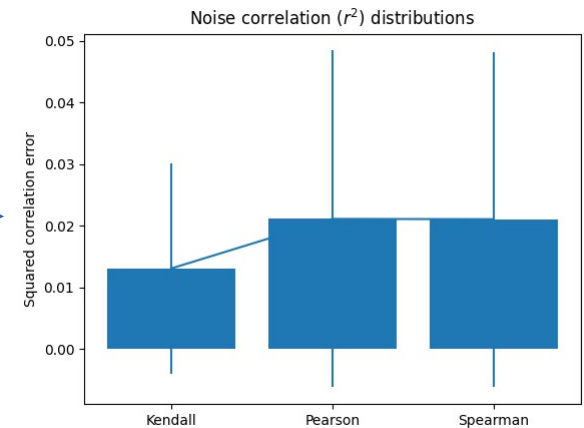
```
## correlation estimation errors under H0
numExprs = 1000
nValues = 50
nCategories = 6
c = np.zeros((numExprs,3))
for i in range(numExprs):
    # create data
    x = np.random.randint(low=0,high=nCategories,size=nValues)
    y = np.random.randint(low=0,high=nCategories,size=nValues)
    # store correlations
    c[i,:] = [ stats.kendalltau(x,y)[0],
              stats.pearsonr(x,y)[0],
              stats.spearmanr(x,y)[0] ]

print(c)
## show the graphs
plt.bar(range(3),np.mean(c**2,axis=0))
plt.errorbar(range(3),np.mean(c**2,axis=0),yerr=np.std(c**2,ddof=1,axis=0))
plt.xticks(range(3),('Kendall','Pearson','Spearman'))
plt.ylabel('Squared correlation error')
plt.title('Noise correlation ($r^2$) distributions')
plt.show()
```

```
warnings.warn("Unable to import Axes3D.",
[[-0.10054765 -0.10865317 -0.12377876]
 [-0.1042392  -0.14381055 -0.1372377 ]
 [-0.14954069 -0.21700241 -0.187386 ]
 ...
 [-0.1032819  -0.12712068 -0.13336634]
 [ 0.04089861  0.0755566  0.06292129]
 [-0.06690613 -0.07272651 -0.09329404]]
atalet@uznem:~/workspace/YZ_EGITIM/ozetler
```

```
plt.plot(c[:100,:], 's-')
plt.xlabel('Experiment number')
plt.ylabel('Correlation value')
plt.legend(('K','P','S'))
plt.show()

plt.imshow(np.corrcoef(c.T),vmin=.9,vmax=1)
plt.xticks(range(3),['K','P','S'])
plt.yticks(range(3),('K','P','S'))
plt.colorbar()
plt.title('Correlation matrix')
plt.show()
```



```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## generate some data!
N = 40
# movie ratings
docuRatings = np.random.randint(low=1,high=6,size=N)
# education level (1-4, correlated with docuRatings)
eduLevel = np.ceil( (docuRatings + np.random.randint(low=1,high=5,size=N) )/9 * 4 )
# compute the correlations
cr = [0,0,0]
cr[0] = stats.kendalltau(eduLevel,docuRatings)[0]
cr[1] = stats.pearsonr(eduLevel,docuRatings)[0]
cr[2] = stats.spearmanr(eduLevel,docuRatings)[0]
# round for convenience
cr = np.round(cr,4)
# plot the data
plt.plot(eduLevel+np.random.randn(N)/30,docuRatings+np.random.randn(N)/30,'ks',markersize=10,markerfacecolor=[0,0,0,.25])
plt.xticks(np.arange(4)+1)
plt.yticks(np.arange(5)+1)
plt.xlabel('Education level')
plt.ylabel('Documentary ratings')
plt.grid()
plt.title('$r_k$ = %g, $r_p$=%g, $r_s$=%g'%(cr[0],cr[1],cr[2]))
plt.show()

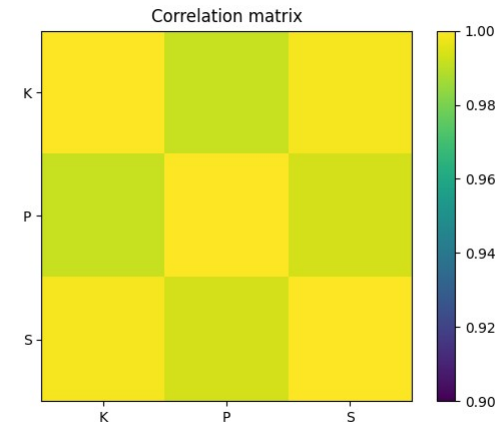
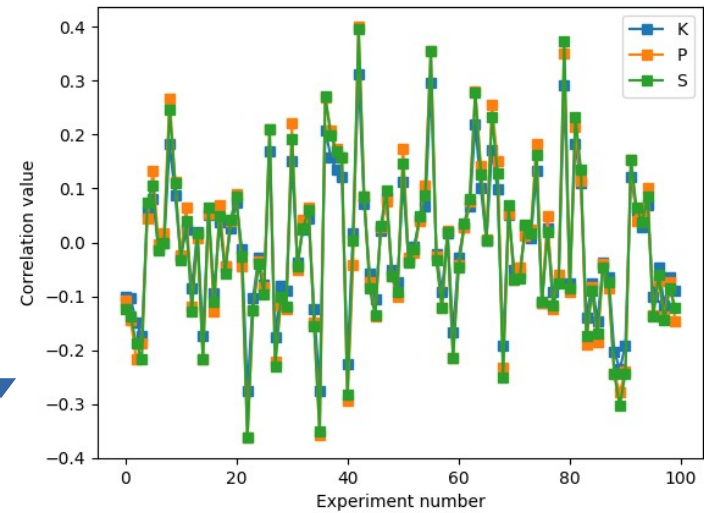
## correlation estimation errors under H0
numExprs = 1000
nValues = 50
nCategories = 6
c = np.zeros((numExprs,3))
for i in range(numExprs):
    # create data
    x = np.random.randint(low=0,high=nCategories,size=nValues)
    y = np.random.randint(low=0,high=nCategories,size=nValues)
    # store correlations
    c[i,:] = [ stats.kendalltau(x,y)[0],
              stats.pearsonr(x,y)[0],
              stats.spearmanr(x,y)[0] ]

print(c)
## show the graphs
plt.bar(range(3),np.mean(c**2,axis=0))
plt.errorbar(range(3),np.mean(c**2,axis=0),yerr=np.std(c**2,ddof=1,axis=0))
plt.xticks(range(3),('Kendall','Pearson','Spearman'))
plt.ylabel('Squared correlation error')
plt.title('Noise correlation ($r^2$) distributions')
plt.show()

plt.plot(c[:100,:],'s-')
plt.xlabel('Experiment number')
plt.ylabel('Correlation value')
plt.legend(('K','P','S'))
plt.show()

plt.imshow(np.corrcoef(c.T),vmin=.9,vmax=1)
plt.xticks(range(3),['K','P','S'])
plt.yticks(range(3),('K','P','S'))
plt.colorbar()
plt.title('Correlation matrix')
plt.show()

```



Cosine vs. Pearson

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

$$\cos(\theta) = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}}$$

```

import matplotlib.pyplot as plt
import numpy as np
from scipy import spatial

# range of requested correlation coefficients
rs = np.linspace(-1,1,100)
# sample size
N = 500
# initialize output matrix
corrs = np.zeros((len(rs),2))
# loop over a range of r values
for ri in range(len(rs)):
    # generate data
    x = np.random.randn(N)
    y = x*rs[ri] + np.random.randn(N)*np.sqrt(1-rs[ri]**2)
    # optional mean-off-centering
    #x = x+10
    #y = y+10
    # compute correlation
    corrs[ri,0] = np.corrcoef(x,y)[0,1]
    # compute cosine similarity
    cs_num = sum(x*y)
    cs_den = np.sqrt(sum(x*x)) * np.sqrt(sum(y*y))
    corrs[ri,1] = cs_num / cs_den
    # using built-in distance function
    #corrs[ri,1] = 1-spatial.distance.cosine(x,y)

## visualize the results
plt.plot(rs,corrs[:,0],'s-',label='Correlation')
plt.plot(rs,corrs[:,1],'s-',label='Cosine sim.')
plt.legend()
plt.xlabel('Requested correlation')
plt.ylabel('Empirical correlation')
plt.axis('square')
plt.show()

plt.plot(corrs[:,0],corrs[:,1],'ks')
plt.axis('square')
plt.xlabel('Correlation')
plt.ylabel('Cosine similarity')
plt.show()

# their empirical correlation
np.corrcoef(corrs.T)

```

